

Airflow at the center of your pipelines

Why deploy a full service when you only need a robust library? Redefining Micro-Orchestration.



The context

Apache Airflow is the backbone of Big Data. But in the era of microservices and Edge Computing, centralized orchestration often introduces unnecessary latency and maintenance complexity. If your pipeline doesn't need thousands of tasks distributed across a Kubernetes cluster, maybe you don't need an orchestrator-as-a-service. You need Embedded Orchestration.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="task", retry_count=3, retry_delay=2)
def task(data):
    return {"ok": run(data["job"])}

pipe = Pipeline(pipeline_name="airflow_like", tracking_db="jobs.db")
pipe.set_steps([task])

# No scheduler daemon, no heavy metadata DB.
```

At a glance

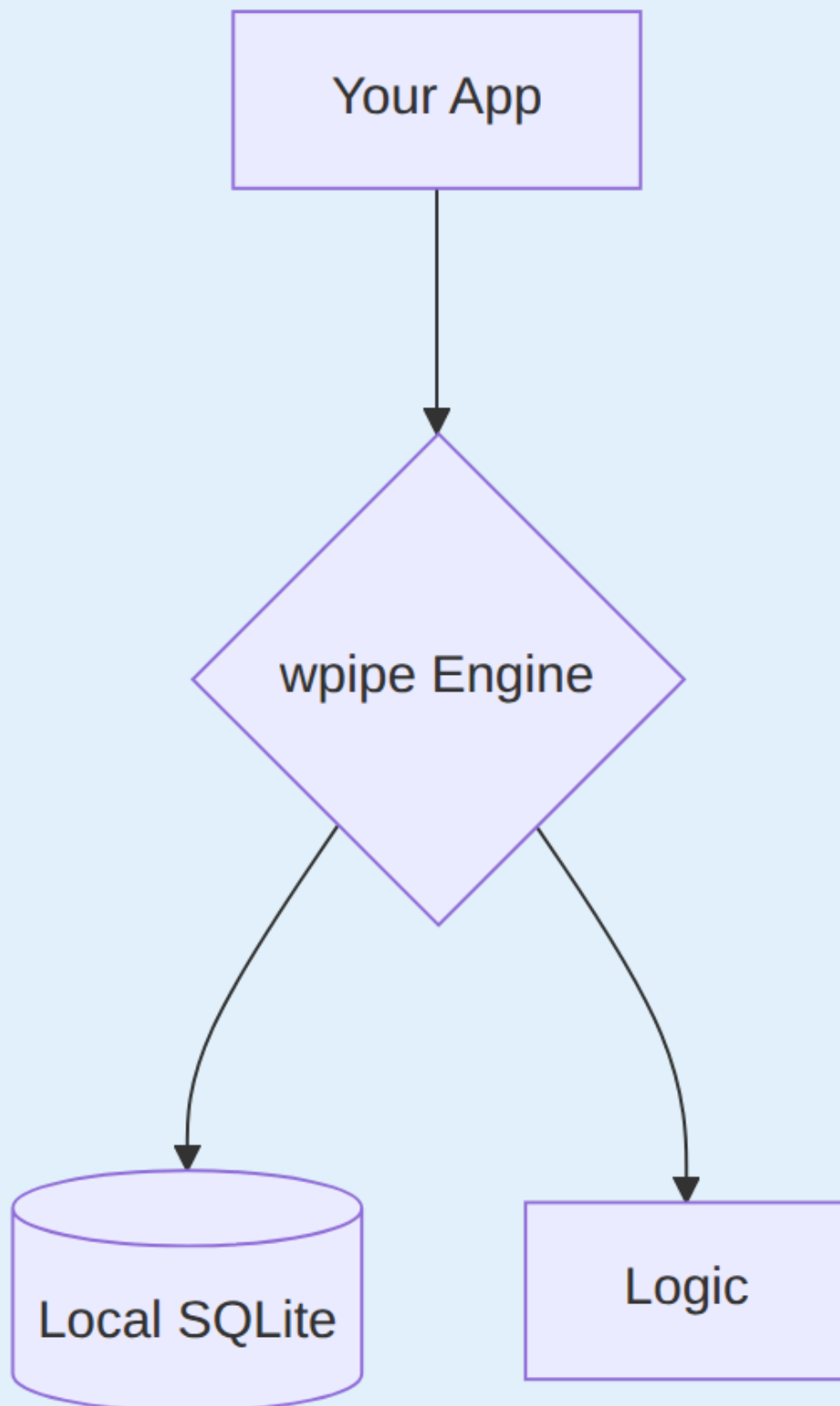
Dimension	Apache Airflow	wpipe
Architecture	Client-Server (Centralized)	Native Library (Decentralized)
Footprint	GBs (Multiple containers)	MBs (Single process)

Dimension	Apache Airflow	wpipe
Persistence	External Postgres/Redis	SQLite WAL (Zero-Config)
Task Latency	Seconds (Scheduling overhead)	Milliseconds (Direct execution)
Maintenance	High (Cluster updates)	None (Dependency management)

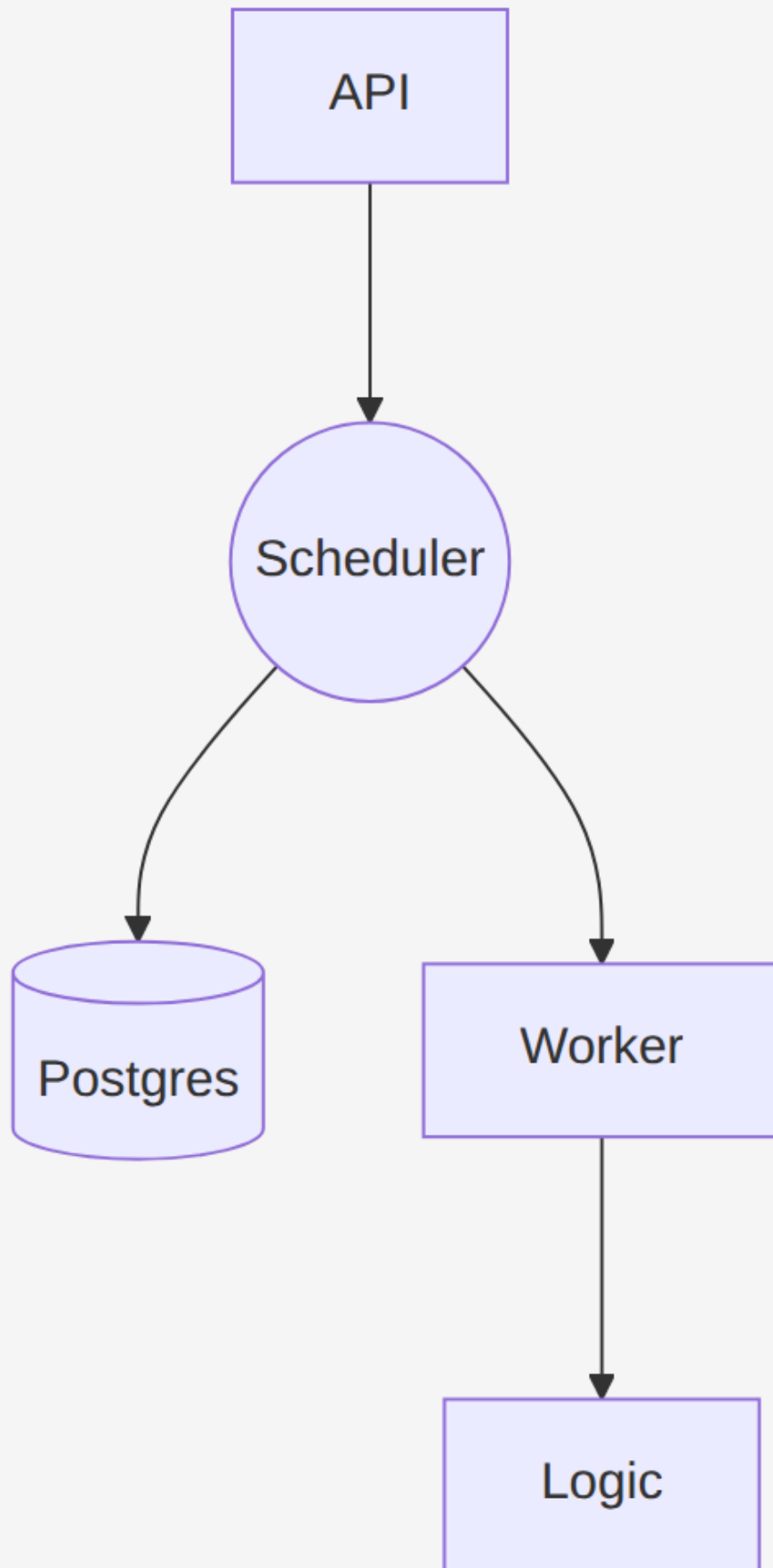
Under the hood

The flow behind the post, as wpipe renders it from code.

wpipe: Native Library



Airflow: External Service



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Airflow** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.